

Reviewer Pack — Minimal Order of Topological Entropy and Resolution Proof Wi...

Entry ccaa1f00c9ac · INCONCLUSIVE

SEC autonomous research engine — attribution: Ludovico Kubler

2026-06-05 08:30:34 UTC

Minimal Order of Topological Entropy and Resolution Proof Width Inequality

Entry ID: ccaa1f00c9ac **Verdict:** INCONCLUSIVE **Recorded:** 2026-06-05 08:30:34 UTC

1. Conjecture

Field A (mathematical branch): Algebraic Topology (Topological Entropy) **Field B** (complexity object): Resolution Proofs (Proof Complexity)

Statement:

For every d -regular graph G , the minimal order of topological entropy ($h(G)$) of its associated Tseitin formula φ_G is linearly correlated with its resolution proof width $w(\varphi_G)$, such that $h(G) = \Omega(w(\varphi_G))$.

Rationale (proposer's reasoning):

Topological entropy measures the complexity of a dynamical system and can be linked to computational complexity through the study of graphs. Resolution proofs, being a proof system, have been shown to be related to the structure of graphs. This conjecture aims to exploit this connection by relating topological entropy with resolution proof width.

Taxonomy category: TopologicalEntropyResolutionProofWidth (status at proposal time:)

2. Pre-registration (Popper-style)

Hash (SHA-256 prefix): 03d0131d82ec19dd

This hash commits to the conjecture statement, acceptance criterion, and seed list **before** the empirical test runs. Tampering with the test or its data after this hash was computed would be detectable.

Acceptance criterion:

The conjecture is supported if the Pearson correlation coefficient of the computed minimal order of topological entropy ($h(G)$) and resolution proof width ($w(\varphi_G)$) for 30 random d -regular graphs with $n \leq 40$ variables exceeds 0.7, and no seed produces a correlation coefficient below 0.5.

3. Barrier filter (F1)

Final: PASS

Barrier	Verdict	Confidence	LLM ₁	LLM ₂
RELATIVIZATION	SAFE	1.00	SAFE	SAFE
NATURAL_PROOFS	SAFE	0.50	UNCERTAIN	UNCERTAIN
ALGEBRIZATION	SAFE	1.00	SAFE	SAFE
KARP_LIPTON	SAFE	0.50	UNCERTAIN	UNCERTAIN

4. Novelty audit

Verdict: NOVEL against 0 hits across arXiv + Semantic Scholar + ECCC

5. Empirical test harness

Multi-seed protocol: 5 seeds (11, 23, 37, 53, 71)

Sandbox: isolated subprocess, stdlib-only, ≤ 90 s wall time per cycle **Execution:** rc=0, elapsed=0.7s

5.1 Generated Python source

```
import random
import math
from fractions import Fraction

def generate_d_regular_graph(n, d):
    if n % d != 0:
        raise ValueError("Graph size must be a multiple of the degree")

    graph = [[] for _ in range(n)]
    edges_added = set()

    def add_edge(u, v):
        if (u, v) not in edges_added and (v, u) not in edges_added:
            graph[u].append(v)
            graph[v].append(u)
            edges_added.add((u, v))
            edges_added.add((v, u))

    for i in range(n):
        for j in range(i + 1, n):
            if len(graph[i]) < d and len(graph[j]) < d:
                add_edge(i, j)

    return graph

def topological_entropy(graph):
    n = len(graph)
    degrees = [len(neighbors) for neighbors in graph]
    max_degree = max(degrees)

    # Approximate the topological entropy using a finite cover
    h_G = 0.0
    for degree in degrees:
        if degree > 1:
            h_G += math.log(degree / (max_degree - 1))

    return h_G
```

```

def resolution_proof_width(graph):
    n = len(graph)
    max_clause_length = 0

    # Construct the Tseitin formula and compute the maximum clause length
    for i in range(n):
        for j in range(i + 1, n):
            if j not in graph[i]:
                max_clause_length = max(max_clause_length, 2)

    return max_clause_length

def run_trial(seed: int) -> dict:
    random.seed(seed)
    d_values = [3, 4, 5, 6] # Degrees to test
    n_max = 0
    h_G_sum = 0.0
    w_phi_G_sum = 0.0
    instances_tested = 0

    for n in range(10, 21): # Test sizes from 10 to 20
        for d in d_values:
            graph = generate_d_regular_graph(n * d, d)
            h_G = topological_entropy(graph)
            w_phi_G = resolution_proof_width(graph)

            if n > n_max:
                n_max = n

            h_G_sum += h_G
            w_phi_G_sum += w_phi_G
            instances_tested += 1

    mean_h_G = h_G_sum / instances_tested
    mean_w_phi_G = w_phi_G_sum / instances_tested
    correlation_coefficient = (instances_tested * sum(h_G * w_phi_G for h_G, w_phi_G in zip([h_G for h_G, w_phi_G in zip(h_G_sum, w_phi_G_sum)], [w_phi_G for h_G, w_phi_G in zip(h_G_sum, w_phi_G_sum)])))

    return {
        "metric_name": "Pearson correlation coefficient",
        "metric_value": correlation_coefficient,
        "instances_tested": instances_tested,
        "n_max": n_max,
        "conjecture_holds": correlation_coefficient >= 0.7,
        "counterexample": "" if correlation_coefficient >= 0.5 else f"Correlation coefficient {correlation_coefficient}"
    }

if __name__ == "__main__":
    import sys
    seeds = [int(seed) for seed in sys.argv[1:]] or [2, 3, 5, 7, 11, 13, 17, 19, 23, 29, 31, 37, 41]

    results = []
    for seed in seeds:
        result = run_trial(seed)
        print(f"TRIAL: {{'seed': {seed}, ...{result}...}}")
        results.append(result)

```

```

mean_metric_value = sum(result["metric_value"] for result in results) / len(results)
support_fraction = sum(1 for result in results if result["conjecture_holds"]) / len(results)

if all(result["conjecture_holds"] for result in results):
    print(f"RESULT: SUPPORTED mean={mean_metric_value} std=0.0 support_fraction=1.0")
elif support_fraction >= 0.8:
    print(f"RESULT: SUPPORTED mean={mean_metric_value} std=0.0 support_fraction={support_fraction}")
else:
    first_failing_seed = next(seed for seed, result in zip(seeds, results) if not result["conjecture_holds"])
    print(f"RESULT: FALSIFIED counterexample=\"{result['counterexample']}\" first_failing_seed={first_failing_seed}")

```

6. Per-seed results

(no seed-level data — test crashed before producing TRIAL: lines)

7. Test stdout (last 2KB)

```

{'seed': 503, ...{'metric_name': 'Pearson correlation coefficient', 'metric_value': 1.0000164038885}
TRIAL: {'seed': 547, ...{'metric_name': 'Pearson correlation coefficient', 'metric_value': 1.0000164038885}
TRIAL: {'seed': 593, ...{'metric_name': 'Pearson correlation coefficient', 'metric_value': 1.0000164038885}
TRIAL: {'seed': 631, ...{'metric_name': 'Pearson correlation coefficient', 'metric_value': 1.0000164038885}
TRIAL: {'seed': 677, ...{'metric_name': 'Pearson correlation coefficient', 'metric_value': 1.0000164038885}
TRIAL: {'seed': 727, ...{'metric_name': 'Pearson correlation coefficient', 'metric_value': 1.0000164038885}
TRIAL: {'seed': 773, ...{'metric_name': 'Pearson correlation coefficient', 'metric_value': 1.0000164038885}
TRIAL: {'seed': 821, ...{'metric_name': 'Pearson correlation coefficient', 'metric_value': 1.0000164038885}
TRIAL: {'seed': 877, ...{'metric_name': 'Pearson correlation coefficient', 'metric_value': 1.0000164038885}
TRIAL: {'seed': 929, ...{'metric_name': 'Pearson correlation coefficient', 'metric_value': 1.0000164038885}
RESULT

```

8. Critic adversarial review

Critic verdict: CHALLENGE

Critic reasoning:

The test code only tests graphs with sizes from 10 to 20 and degrees from 3 to 6, which is a very limited range. The conjecture's validity may depend on the specific properties of these graphs, and the results might not generalize to all d-regular graphs.

9. Final verdict & safety rail

Verdict: INCONCLUSIVE

Reasoning:

The test code only considers a limited range of graph sizes and degrees, which may not be representative of all d-regular graphs. The critic's challenge raises concerns about the generalizability of the results. | next: Expand the test to include a wider range of graph sizes and degrees to verify the conjecture's validity across different types of d-regular graphs.

11. Audit log (LLM calls)

Total LLM calls: 10

#	Phase	Provider	Model	Tokens in	out	Latency (ms)
1	propose	ollama_remote	glm4:latest	0	0	15848
2	preregistration	ollama_remote	glm4:latest	0	0	9466
3	novelty	ollama_remote	glm4:latest	0	0	8995
4	novelty	ollama_remote	glm4:latest	0	0	8765
5	test_gen	ollama_remote	qwen2.5-coder:7b	0	0	51423
6	test_gen	ollama_remote	qwen2.5-coder:7b	0	0	11314
7	test_gen	ollama_remote	qwen2.5-coder:7b	0	0	12898
8	test_gen	ollama_remote	qwen2.5-coder:7b	0	0	13946
9	critic	ollama_remote	glm4:latest	0	0	17660
10	judge	ollama_remote	glm4:latest	0	0	10004

Totals: 0 input tokens, 0 output tokens, ~\$0.0000 cost, 160320 ms total latency. Provider mix: {'ollama_remote': 10}

(full prompt+response transcripts available in *research/audit/ccaa1f00c9ac.jsonl*)

12. Reproducibility

To reproduce this cycle:

1. Apply the test harness in section 5.1 with seeds 11,23,37,53,71
2. The Python sandbox is pure-stdlib; any Python 3.11+ should suffice
3. The full LLM transcripts are in *research/audit/ccaa1f00c9ac.jsonl* for verification of provenance
4. The pre-registration hash in section 2 commits to the test design before execution; tampering would be detectable.

Sandbox file: *research/pvsnp_sandbox/test_*.py* (per-cycle)

Replay tarball: *research/replay/ccaa1f00c9ac.tar.gz* (if generated)